

SQL subqueries are queries nested inside another query to perform operations like filtering, aggregation, and data retrieval. Below are the main types of subqueries, along with SQL queries and sample data.

1. Single-Row Subquery

A subquery that returns a **single value** (one row, one column). Used with comparison operators (`=`, `>`, `<`, `>=`, `<=`, `<>`).

Example: Find employees earning more than the average salary

Sample Table: `employees`

emp_id	name	department	salary
1	Alice	IT	5000
2	Bob	HR	4000
3	Charlie	IT	7000
4	David	Finance	5500
5	Eve	HR	4500

Query:

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Output:

name	salary
Charlie	7000
David	5500

The subquery calculates the average salary, and the outer query filters employees earning above that.

2. Multi-Row Subquery

Returns **multiple rows** for a **single column**. Used with `IN`, `ANY`, `ALL`.

Example: Find employees working in the same department as 'Bob'

Query:

```
SELECT name
```

```
FROM employees
WHERE department IN (SELECT department FROM employees WHERE name =
'Bob');
```

OUTPUT:

name
Bob
Eve

The subquery finds Bob's department (HR), and the outer query fetches all employees from HR.

3. Multi-Column Subquery

Returns multiple columns and is used for comparing multiple values.

Example: Find employees with the same job and department as 'Alice'

Sample Table: employees_extended

emp_id	name	department	job_role	salary
1	Alice	IT	Developer	5000
2	Bob	HR	Manager	4000
3	Charlie	IT	Developer	7000
4	David	Finance	Analyst	5500
5	Eve	HR	Manager	4500

Query:

```
SELECT name
FROM employees_extended
WHERE (department, job_role) =
      (SELECT department, job_role FROM employees_extended WHERE name
= 'Alice');
```

OUTPUT:

name
Alice
Charlie

The subquery returns Alice's job (**Developer**) and department (**IT**), and the outer query retrieves employees with the same role and department.

4. Correlated Subquery

A subquery that runs once **for each row** in the outer query.

Example: Find employees earning above their department's average salary

Query:

```
SELECT name, salary
FROM employees e1
WHERE salary > (SELECT AVG(salary)
                FROM employees e2
                WHERE e1.department = e2.department);
```

OUTPUT:

name	salary
Charlie	7000
David	5500

The subquery calculates the **average salary per department**, and the outer query selects employees earning above their department's average.

5. Subquery in **FROM** Clause (Derived Table)

Treats a subquery as a **temporary table**.

Example: Find departments with an average salary above 5000

SQL:

```
SELECT department
FROM (SELECT department, AVG(salary) AS avg_salary
      FROM employees
      GROUP BY department) AS dept_avg
WHERE avg_salary > 5000;
```

OUTPUT:

department
IT
Finance

The subquery calculates department-wise **average salaries**, and the outer query filters those above **5000**.

6. EXISTS/NOT EXISTS Subquery

Checks whether a subquery **returns any rows**.

Example: Find departments that have employees

Sample Table: **departments**

department
IT
HR
Finance
Sales

SQL:

```
SELECT department
FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE d.department =
e.department);
```

OUTPUT:

department
IT
HR
Finance

The **EXISTS** subquery ensures that only departments **having employees** are returned.

Summary of SQL Subquery Types

Subquery Type	Returns	Used With	Example Query
Single-Row	1 row, 1 column	=, >, <, >=, <=	Employees earning above average salary
Multi-Row	Multiple rows, 1 column	IN, ANY, ALL	Employees in Bob's department
Multi-Column	Multiple rows, multiple columns	=, IN	Employees with same job and department as Alice
Correlated	Executes per row	EXISTS, NOT EXISTS, Comparisons	Employees earning above department average
Subquery in FROM	Acts as a temporary table	SELECT, GROUP BY, HAVING	Departments with high average salary
EXISTS/NOT EXISTS	Boolean (TRUE/FALSE)	EXISTS, NOT EXISTS	Departments with employees